**UNIVERSIDAD DEL VALLE DE GUATEMALA**

Data Science
Sección – 20
Ing. Mario Barrientos




# Reconocimiento automático de deletreo manual en American Sign Language (ASL) mediante aprendizaje profundo

*María José Girón Isidro - 23559*
*Daniela Ramírez de León - 23053*
*Leonardo Dufrey Mejía Mejía - 23648*
*Cindy Mishelle Gualim Perez - 21226*
*José Pablo Donado López - 22684*

**GUATEMALA, 19 de octubre de 2026**

**UNIVERSIDAD DEL VALLE DE GUATEMALA**

**ÍNDICE**

INTRODUCCIÓN ..... 3
OBJETIVOS ..... 4
Objetivo general ..... 4
Objetivos específicos ..... 4
MARCO TEÓRICO ..... 5
Relación con el procesamiento del lenguaje natural ..... 6
Redes convolucionales temporales ..... 7
Redes recurrentes, LSTM y GRU ..... 7
Transformers y mecanismo de atención ..... 7
Clasificación temporal conexionista ..... 7
Métricas de evaluación ..... 8
METODOLOGÍA ..... 8
Obtención y exploración de los datos ..... 8
Limpieza y construcción de características ..... 8
Manejo de longitudes y etiquetas ..... 9
Selección de entrenamiento, validación y prueba ..... 9
Selección de algoritmos ..... 9
Entrenamiento y ajuste ..... 10
Herramientas y recursos de cómputo ..... 10
RESULTADOS Y ANÁLISIS DE RESULTADOS ..... 11
Características del conjunto original ..... 11
Limpieza y calidad de los datos ..... 12
Análisis exploratorio ..... 13
Ajuste del modelo CNN-BiGRU ..... 15
Ajuste del Transformer ..... 16
Comparación de los modelos ..... 17
Análisis de errores ..... 18
Aplicación desarrollada ..... 20
CONCLUSIONES ..... 23
REFERENCIAS ..... 25

## INTRODUCCIÓN

El reconocimiento automático del deletreo manual constituye una aplicación del aprendizaje profundo (Deep Learning) orientada a convertir movimientos de las manos en texto. En en el lenguaje de señas americana, conocido como ASL por sus siglas en inglés, el deletreo manual permite representar palabras mediante una secuencia de letras. Su reconocimiento requiere considerar tanto la configuración de la mano como su evolución temporal, debido a que las transiciones entre letras y las diferencias en la ejecución pueden dificultar la identificación de cada carácter.

El presente proyecto aborda la transcripción de secuencias de ASL de un conjunto de datos del American Sign Language Fingerspelling Recognition. Las entradas están representadas mediante puntos de referencia anatómicos, denominados landmarks, obtenidos de grabaciones de video. Estos puntos contienen la información de las posiciones de las manos, el rostro y el cuerpo mediante coordenadas numéricas. Por lo tanto, el trabajo presente le da foco al procesamiento de secuencias de coordenadas y no en el entrenamiento de un detector sobre las imágenes.

El problema consiste en transformar una secuencia de movimientos de longitud variable en una cadena de caracteres, sin disponer de una correspondencia explícita entre cada cuadro y cada letra. Ahora, la mayor dificultad es la ausencia de detecciones, las diferencias entre participantes, desbalance de caracteres y distintas duraciones de secuencias, tal como se observó durante el análisis exploratorio. Estas condiciones hicieron necesario limpiar, normalizar y evaluar varios modelos para generalizar a personas distintas de las utilizadas durante el entrenamiento, que son personas de tono de piel similar en una iluminación que sí era contrastante entre videos.

Además, se resume la caracterización de los datos, el preprocesamiento requerido, la metodología experimental y los resultados disponibles de cuatro arquitecturas complementarias: GRU bidireccional (CNN 1D), GRU, LSTM y un encoder TRANSFORMER.

Como hallazgos principales, es relevante mencionar que el modelo de mayor complejidad computacional es CNN-BiGRU y se sugiere que por ello presentó el menor CER, pero no se considera el mejor al ponderar el rendimiento por complejidad paramétrica, de tiempo y almacenamiento, cuando Transformer conservó un error relativamente cercano a CNN-BiGRU sin desequilibrar otras medidas de complejidad.

## OBJETIVOS

### Objetivo general

Desarrollar y evaluar un prototipo de transcripción automática del deletreo manual en ASL a partir de secuencias de landmarks, mediante técnicas de preprocesamiento y modelos de aprendizaje profundo, considerando su capacidad de ser generalizados a participantes que no fueron observados durante el entrenamiento.

### Objetivos específicos

1. Caracterizar la estructura, distribución y calidad del conjunto de datos, identificando valores faltantes, diferencias entre participantes y desbalance en la frecuencia de los caracteres.
2. Implementar un procedimiento de limpieza y transformación que permita utilizar secuencias de longitud variable, normalizar las coordenadas anatómicas y representar explícitamente la ausencia de detecciones.
3. Establecer una división reproducible de entrenamiento, validación y prueba por participante, evitando que una misma persona aparezca en diferentes particiones.
4. Implementar y ajustar modelos de aprendizaje profundo capaces de producir transcripciones sin requerir una alineación previa entre cuadros y caracteres.
5. Comparar los modelos mediante la tasa de error por carácter, el número de parámetros, los tiempos registrados y los patrones de error.
6. Integrar los resultados en una aplicación que permita consultar transcripciones, procesar archivos compatibles y visualizar el desempeño de los modelos disponibles.

## MARCO TEÓRICO

El deletreo manual consiste en representar letras mediante configuraciones y movimientos de la mano. Su reconocimiento automático se diferencia de la clasificación de una imagen aislada porque la información necesaria para identificar un carácter puede estar distribuida entre varios cuadros.

La coarticulación describe la influencia que ejercen los movimientos anteriores y posteriores sobre la ejecución de una letra. Durante una secuencia continua, las manos pasan de una configuración a otra sin que necesariamente exista una separación claramente observable. Por ello, un modelo debe aprender patrones temporales y considerar el contexto de los movimientos.

El alcance del proyecto es la transcripción de deletreo manual. No comprende la traducción general de ASL, que requeriría abordar vocabulario, gramática y componentes lingüísticos adicionales.

Por otro lado, el procesamiento de imágenes permite extraer información de representaciones visuales. En aplicaciones de reconocimiento de movimientos, una alternativa al análisis directo de píxeles es utilizar puntos anatómicos detectados previamente. Esta representación transforma una imagen en un conjunto de coordenadas que puede analizarse como una serie temporal.

MediaPipe Hands utiliza un procedimiento de detección de la palma y estimación de puntos de la mano. Cada mano se representa mediante 21 landmarks, correspondientes a la muñeca y diferentes articulaciones de los dedos. En el conjunto utilizado también se incluyen puntos del rostro y de la pose corporal.

Trabajar con coordenadas reduce la dimensionalidad de entrada frente al procesamiento de imágenes completas. Sin embargo, no elimina los problemas originados durante la captura: la iluminación, la oclusión o el movimiento pueden afectar la detección y producir valores faltantes. Asimismo, las coordenadas constituyen estimaciones geométricas y no deben interpretarse automáticamente como medidas físicas expresadas en metros.

La especialidad relacionada con el proyecto es el reconocimiento de gestos y la accesibilidad comunicativa. No se realiza análisis médico ni diagnóstico a partir de las imágenes.

**Relación con el procesamiento del lenguaje natural**

El procesamiento del lenguaje natural estudia métodos computacionales para representar, analizar y generar lenguaje humano. En este proyecto, su relación principal se encuentra en la representación de la salida como una secuencia de caracteres y en la comparación entre transcripciones.

Las etiquetas se codifican a nivel de carácter, conservando letras, números, espacios y signos. No se aplican eliminación de palabras vacías, lematización ni stemming, debido a que el objetivo es reproducir exactamente el texto de referencia. Estas transformaciones modificarían las etiquetas y eliminarían información necesaria para evaluar la transcripción.

Aunque uno de los modelos utiliza una arquitectura Transformer, la implementación disponible no incorpora un modelo de lenguaje externo ni un corrector ortográfico. La atención opera sobre las representaciones de los movimientos.

La normalización busca reducir variaciones asociadas con la ubicación y escala de la persona en la imagen. Para las manos, las coordenadas pueden expresarse respecto a la muñeca y dividirse entre una distancia anatómica de referencia. En la pose corporal puede utilizarse el punto medio de los hombros como origen y su separación como escala.

Los valores faltantes requieren un tratamiento específico. El relleno hacia adelante, o forward-fill, conserva el último valor disponible de cada coordenada dentro de una secuencia. Es una aproximación que permite mantener una entrada numérica continua, pero no recupera el movimiento perdido. En intervalos prolongados puede representar una posición estática que no refleja lo ocurrido.

Por esta razón, el proyecto complementa el relleno con indicadores de detección de cada mano. Estos indicadores permiten distinguir entre información observada e información imputada.

### Redes convolucionales temporales

Las redes convolucionales de una dimensión aplican filtros sobre el eje temporal. Cada filtro combina información de cuadros cercanos para detectar patrones locales de movimiento.

El parámetro stride determina el desplazamiento del filtro. Un stride mayor reduce la longitud de la representación y disminuye el costo de las capas posteriores. No obstante, una reducción excesiva puede eliminar información necesaria para distinguir movimientos breves o producir suficientes pasos de salida para la transcripción.

### Redes recurrentes, LSTM y GRU

Las redes recurrentes procesan secuencias manteniendo un estado que resume información de pasos anteriores, como si tuviese un historial. Las variantes LSTM y GRU incorporan mecanismos de compuertas para controlar qué información se conserva y actualiza.

Una red bidireccional procesa la secuencia en ambos sentidos y combina las representaciones obtenidas. Esto permite utilizar contexto anterior y posterior, lo cual resulta pertinente para el reconocimiento de movimientos continuos. Sin embargo, requiere disponer del contexto futuro, por lo que la implementación evaluada corresponde al procesamiento de secuencias completas.

En el proyecto se implementó una GRU bidireccional combinada con convoluciones temporales. Este modelo de Deep Learning permite extraer lo mejor de dos universos: la memoria contextual a largo plazo de las redes recurrentes y la propiedad de extraer características locales de las convoluciones.

### Transformers y mecanismo de atención

Los Transformers utilizan mecanismos de atención para combinar información de distintas posiciones de una secuencia. En lugar de depender exclusivamente de un estado recurrente, cada posición puede relacionarse directamente con otras posiciones relevantes.

La codificación posicional incorpora información sobre el orden en el tiempo. La atención de múltiples cabezas permite aprender distintas relaciones entre las representaciones. En la atención completa, el costo asociado a las interacciones entre posiciones crece cuadráticamente con la longitud de la secuencia, lo que motiva su reducción previa mediante convoluciones.

### Clasificación temporal conexionista

Connectionist Temporal Classification, o CTC, permite entrenar modelos cuando se conoce la transcripción completa, pero no la ubicación temporal de cada carácter. Introduce un símbolo adicional, denominado blank, y considera distintos alineamientos compatibles con la misma etiqueta.

Durante la decodificación utilizada en el proyecto se selecciona el símbolo más probable en cada paso, se colapsan repeticiones consecutivas y se eliminan los símbolos blank. De esta

manera, una salida temporal extensa se convierte en una cadena de caracteres. Este procedimiento se denomina decodificación greedy.

La longitud de salida debe ser suficiente para representar la etiqueta y separar caracteres consecutivos repetidos. Esta condición limita cuánto puede reducirse la secuencia mediante stride.

### Métricas de evaluación

La métrica principal es la tasa de error por carácter, o CER:

CER = (sustituciones + eliminaciones + inserciones) / número de caracteres de referencia.

Un CER menor representa una mejor transcripción. Su valor puede superar uno cuando se producen numerosas inserciones.

El proyecto también registra el complemento 1 − CER. Sin embargo, este valor no debe interpretarse como el porcentaje de frases completamente correctas ni como una exactitud convencional de clasificación, porque se deriva de operaciones de edición.

La evaluación se complementa con tiempos de procesamiento, número de parámetros y errores por carácter. Estos indicadores permiten comparar la calidad de las transcripciones y los recursos necesarios para obtenerlas.

## METODOLOGÍA

### Obtención y exploración de los datos

Se utilizó el conjunto American Sign Language Fingerspelling Recognition, organizado mediante un archivo de metadatos y archivos Parquet con coordenadas por cuadro. Primero se revisaron la estructura de los archivos, los identificadores, las frases objetivo y la distribución de participantes.

Debido al tamaño del conjunto completo, el procesamiento de landmarks y el modelado se realizaron con dos archivos seleccionados. Estos contenían 1,997 secuencias antes del filtrado. El análisis de las frases y de la participación global utilizó las 67,208 filas del archivo de metadatos.

Esta distinción es necesaria: las estadísticas de duración y detección obtenidas en la muestra no representan una medición exhaustiva de todos los landmarks del conjunto original.

### Limpieza y construcción de características

Se verificó la existencia de cuadros duplicados y se cuantificaron los valores faltantes por región anatómica. Posteriormente, se conservaron las manos y la pose corporal y se descartaron los puntos del rostro para reducir la dimensionalidad.

Se normalizó cada mano respecto a la muñeca y la pose respecto al punto medio de los hombros. Las escalas se calcularon mediante distancias anatómicas, utilizando la mediana de la secuencia para reducir la influencia de cuadros extremos.

Las coordenadas faltantes se completaron mediante forward-fill dentro de cada secuencia. Los valores que permanecían ausentes al inicio se sustituyeron por cero después de la normalización. También se añadieron dos indicadores de detección, uno por mano.

Se descartaron las secuencias con al menos una mano detectada en menos del 10 % de sus cuadros. El procedimiento excluyó 119 secuencias y conservó 1,878 para el modelado.

La representación final contiene 227 características por cuadro: 225 coordenadas correspondientes a 75 puntos anatómicos y dos indicadores de detección.

**Manejo de longitudes y etiquetas**

Las secuencias con más de 384 cuadros se submuestrearon mediante posiciones equiespaciadas. Esta transformación afectó a 38 secuencias, aproximadamente el 2 % de las conservadas. Las secuencias más cortas mantuvieron su longitud y se rellenaron temporalmente al formar lotes.

Las etiquetas se codificaron mediante 59 caracteres. Se reservó el índice cero para el símbolo blank, por lo que los modelos generan probabilidades sobre 60 clases.

**Selección de entrenamiento, validación y prueba**

La división se realizó por participante, con semilla 42 y proporciones objetivo de 70 %, 15 % y 15 %. Ningún participante aparece en más de una partición.

| Partición | Participantes | Secuencias |
|---|---|---|
| Entrenamiento | 64 | 1,315 |
| Validación | 14 | 295 |
| Prueba | 14 | 268 |
| Total | 92 | 1,878 |

Separar por participante permite evaluar la generalización a personas no observadas durante el entrenamiento. Una separación aleatoria por secuencia podría producir resultados más favorables si el modelo reconociera características particulares de personas presentes en ambas particiones.

El conjunto de validación se utilizó para seleccionar configuraciones. El conjunto de prueba se reservó para evaluar los modelos seleccionados.

**Selección de algoritmos**

Se implementaron dos modelos con enfoques diferentes de procesamiento temporal.

El modelo CNN-BiGRU combina convoluciones para detectar patrones locales y reducir la longitud con una GRU bidireccional para incorporar contexto temporal. Se seleccionó por su adecuación al reconocimiento de movimientos y por permitir combinar información local y contextual.

El Transformer combina convoluciones iniciales con atención y codificación posicional. Se eligió para estudiar una alternativa capaz de relacionar directamente distintas partes de la secuencia.

Ambos modelos utilizan la misma representación de entrada, división de datos y función de pérdida CTC. Esto permite comparar sus resultados bajo un protocolo común.

**Entrenamiento y ajuste**

Los experimentos utilizaron AdamW, una programación de tasa de aprendizaje OneCycleLR, recorte de gradiente y un máximo de 60 épocas. Se configuró parada temprana con paciencia de 10 épocas y se conservaron los pesos correspondientes al menor CER de validación.

El ajuste comparó variantes respecto a una configuración base. Se modificaron aspectos como capacidad, dropout, stride, profundidad y tasa de aprendizaje.

El repositorio incluye un módulo de aumentación con transformaciones geométricas y temporales. Sin embargo, los notebooks de las dos corridas reportadas crean los cargadores sin activar estas transformaciones. Por lo tanto, los resultados actuales corresponden a entrenamiento sin aumentación y su efecto permanece pendiente de evaluación.

**Herramientas y recursos de cómputo**

Se utilizó Python como lenguaje principal. NumPy y pandas facilitaron la manipulación de datos; PyArrow permitió leer los archivos Parquet; Matplotlib y seaborn se emplearon para las visualizaciones; y SciPy apoyó los análisis estadísticos.

PyTorch se utilizó para implementar las redes, calcular la pérdida CTC y ejecutar el entrenamiento. Jupyter permitió organizar los experimentos y conservar sus salidas. Git y GitHub se utilizaron para versionar el trabajo.

La aplicación se desarrolló con Streamlit y Plotly, aprovechando la posibilidad de reutilizar directamente los componentes escritos en Python.

Los archivos de métricas de ambos modelos indican ejecución en CPU. No se dispone de una especificación completa y uniforme del procesador y la memoria utilizados, por lo que los tiempos deben interpretarse como mediciones de las ejecuciones registradas.

## RESULTADOS Y ANÁLISIS DE RESULTADOS

### Características del conjunto original

El archivo de metadatos contiene 67,208 secuencias de 94 participantes. Sus cinco variables originales son path, file_id, sequence_id, participant_id y phrase. Los campos numéricos de identificación se interpretan como identificadores, no como magnitudes continuas.

Las frases contienen entre 1 y 31 caracteres, con media de 17.8 y mediana de 17. La participación es altamente variante: se registran entre 1 y 1,535 secuencias por persona, con mediana de 794.5.

Cada cuadro contiene 543 puntos anatómicos: 468 del rostro, 33 de pose y 21 por cada mano. Al considerar tres coordenadas por punto se obtienen 1,629 variables de coordenadas, más la columna frame, para un total de 1,630 columnas.

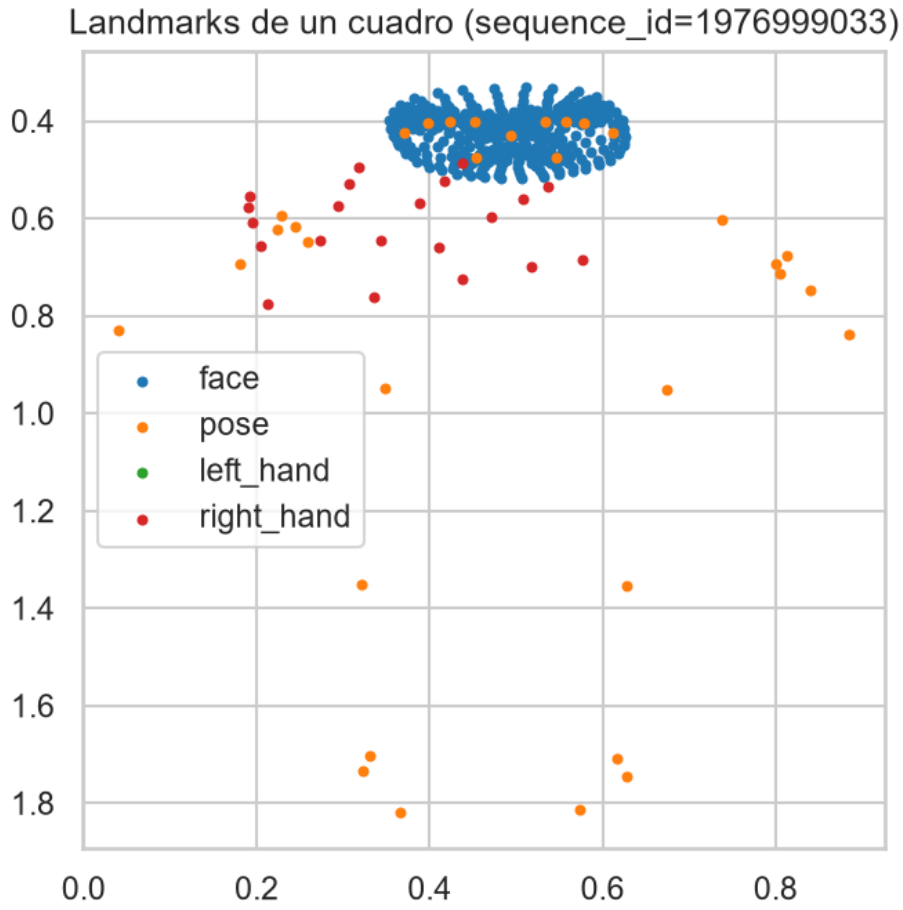


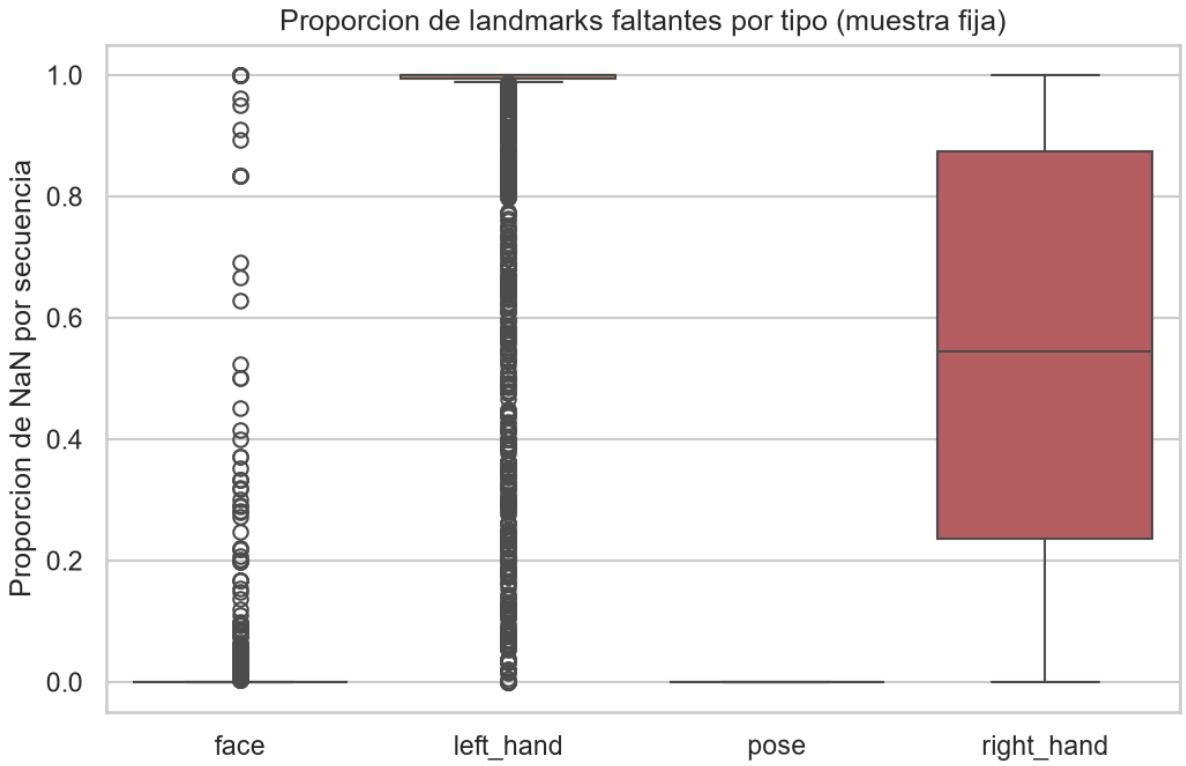

Figura 1. Distribución espacial de los landmarks de un cuadro. La representación permite identificar las regiones anatómicas disponibles como entradas del análisis.

**Limpieza y calidad de los datos**

Las salidas del notebook de limpieza, calculadas sobre las 1,997 secuencias de la muestra, muestran las siguientes proporciones medias de valores faltantes por región:

| Región | Proporción media de valores faltantes |
|---|---|
| Rostro | 1.54 % |
| Mano izquierda | 90.19 % |
| Pose corporal | 0.00 % |
| Mano derecha | 54.33 % |

Estos porcentajes corresponden a promedios por secuencia. La ausencia frecuente de una mano no implica necesariamente que toda la secuencia carezca de información, porque la otra puede estar detectada. Por esa razón, el criterio de exclusión considera la presencia de al menos una mano.



La eliminación del rostro retiró 1,404 columnas de coordenadas, equivalentes al 86.13 % de las columnas originales. Se conservaron 226 columnas, incluida frame, antes de construir las 227 características del modelo.

Esta decisión redujo el costo de procesamiento y concentró la entrada en manos y pose. No obstante, no se realizó una comparación experimental con y sin rostro, por lo que no puede afirmarse que su exclusión carezca de efecto sobre la calidad de la transcripción.

**Análisis exploratorio**

La longitud de las secuencias de la muestra presenta una media de 160.81 cuadros, mediana de 147 y desviación estándar de 86.49. El mínimo es de 3 cuadros y el máximo de 751. La distribución tiene asimetría positiva, con una cola de secuencias especialmente largas.

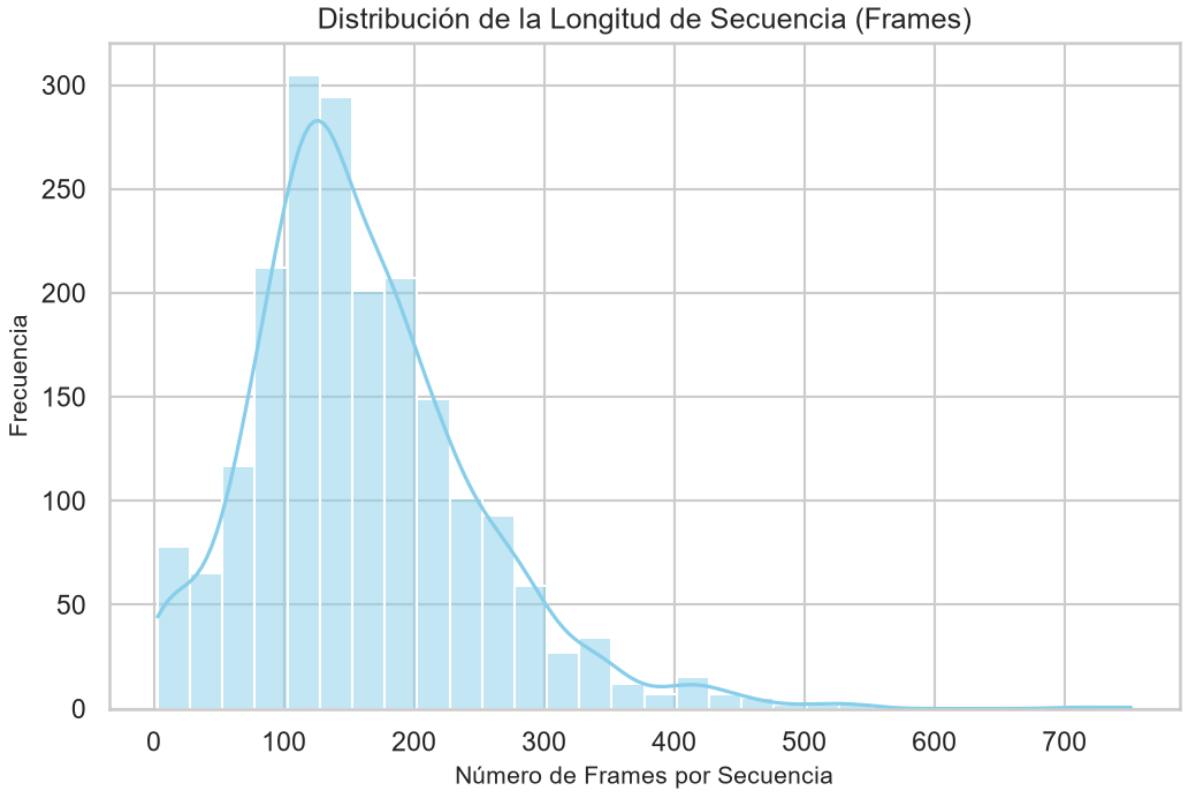



Figura 3. Distribución de la longitud de las secuencias. La cola hacia valores elevados justifica el manejo de longitudes variables y el control del costo temporal.

La distribución de las frases es más acotada, con concentración entre 12 y 22 caracteres para el 50 % central de las observaciones.

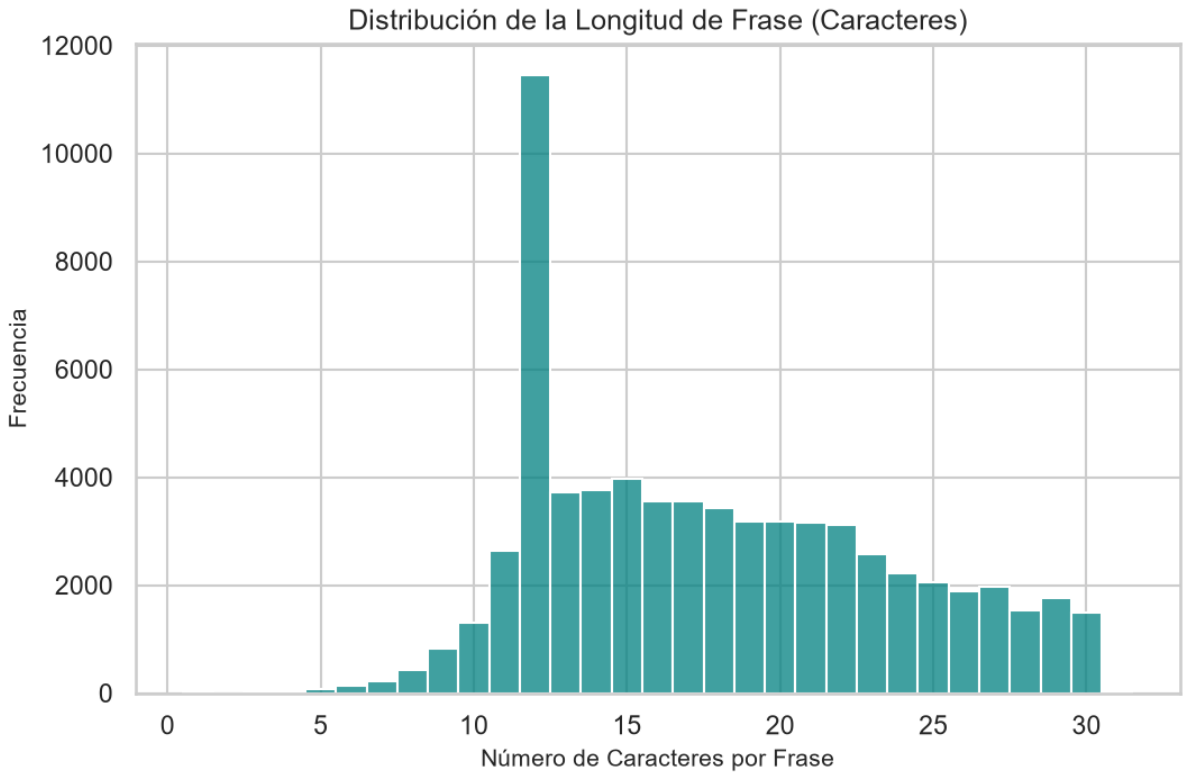



Figura 4. Distribución de la longitud de las frases del archivo de metadatos.

En el análisis bivariado realizado sobre un archivo de 998 secuencias, la correlación de Pearson entre longitud de frase y número de cuadros fue de aproximadamente 0.598. La asociación indica que las frases más largas tienden a requerir más cuadros, pero la dispersión impide asumir una cantidad fija de cuadros por carácter.

La relación entre longitud de secuencia y el indicador de faltantes fue negativa y débil, cercana a −0.188. Este resultado describe una asociación y no demuestra que la pérdida de detecciones cause secuencias más cortas.

También se detectaron 28 secuencias atípicas entre las 998 examinadas mediante el criterio de 1.5 veces el rango intercuartílico. Su duración elevada no constituye por sí sola evidencia de un error de captura, por lo que debe distinguirse entre valores extremos legítimos y observaciones inválidas.

El análisis de las etiquetas identificó 59 caracteres distintos. Las letras representan aproximadamente el 61.2 % de las apariciones; los dígitos, el 24.6 %; los espacios, el 4.9 %; y otros símbolos, el 9.3 %. Esta distribución refleja la presencia de nombres, direcciones, teléfonos y otros textos con componentes numéricos.

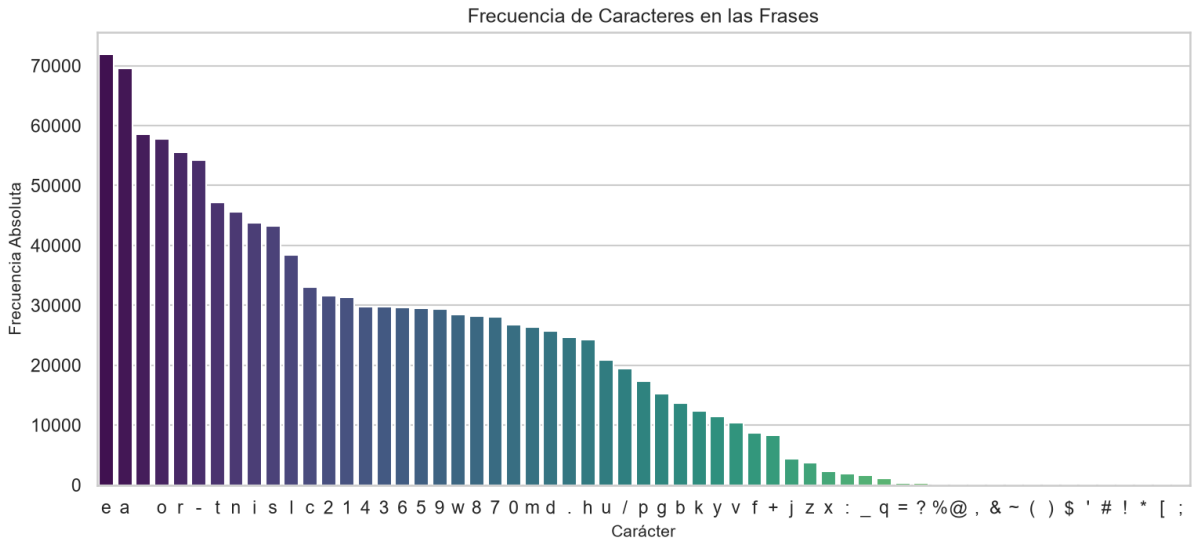



Figura 5. Frecuencia de los caracteres en las etiquetas. El desbalance motiva la evaluación individual de caracteres además del CER global.

**Ajuste del modelo CNN-BiGRU**

Se compararon cuatro variantes del modelo:

| **Variante** | **Cambio principal** | **CER de validación** | **Tiempo de entrenamiento** |
|---|---|---|---|
| Base | 192 canales, 128 unidades GRU, dropout 0.3 | 0.596 | 48.4 min |
| Mayor capacidad | 256 canales y 192 unidades GRU | 0.552 | 47.7 min |
| Stride 4 | Mayor reducción temporal | 0.554 | 14.8 min |
| Dropout 0.5 | Mayor regularización | 0.797 | 33.1 min |

La variante de mayor capacidad obtuvo el menor CER de validación y fue seleccionada. Su configuración utiliza dos capas GRU bidireccionales, kernel de tamaño 5, stride 2 y dropout 0.3, con 1,696,060 parámetros entrenables.

La variante con stride 4 alcanzó un error cercano al de la seleccionada y redujo considerablemente el tiempo registrado. En cambio, aumentar el dropout a 0.5 empeoró el desempeño bajo el presupuesto de entrenamiento utilizado.

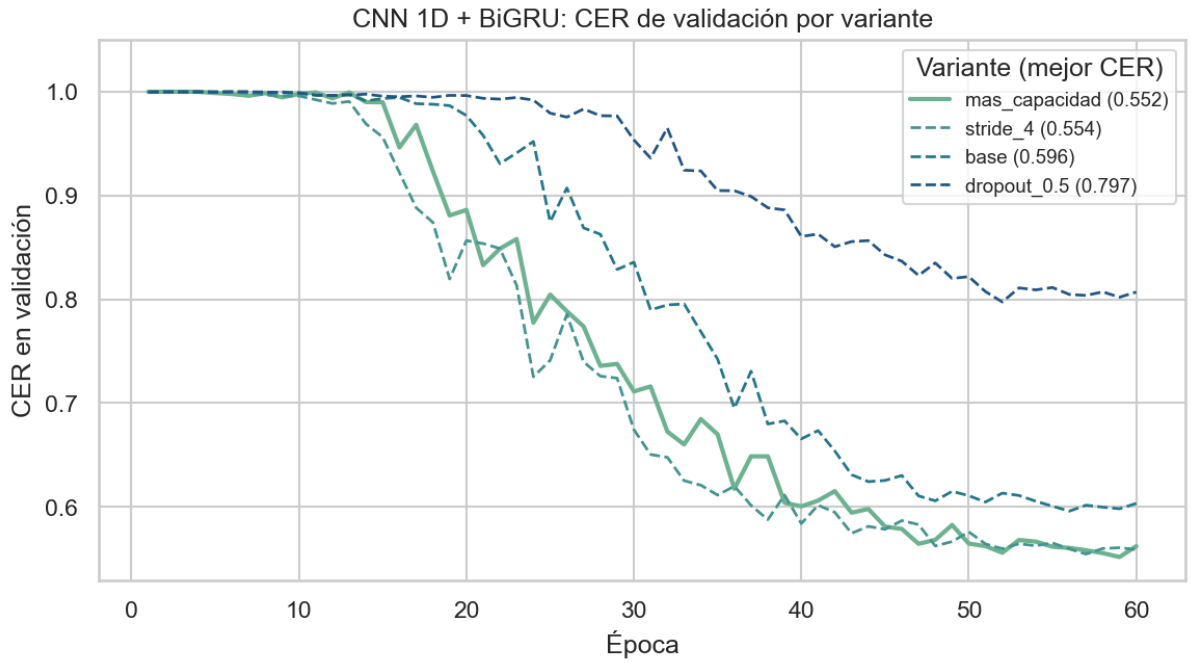



Figura 6. Comparación de las variantes CNN-BiGRU según error de validación y costo de entrenamiento.

**Ajuste del Transformer**

Se evaluaron seis configuraciones:

| **Variante** | **Cambio principal** | **CER de validación** | **Tiempo de entrenamiento** |
|---|---|---|---|
| Base | Dimensión 128, 3 capas, dropout 0.3 | 0.680 | 19.8 min |
| Dropout 0.1 | Menor dropout | 0.584 | 17.8 min |
| Stride 4 | Mayor reducción temporal | 0.588 | 7.5 min |
| Mayor ancho | Dimensión 192 | 0.626 | 19.6 min |
| Tasa de aprendizaje 0.003 | Aumento de la tasa máxima | 0.640 | 19.2 min |
| Mayor profundidad | 6 capas | 0.654 | 36.0 min |

La configuración con dropout 0.1 produjo el menor CER de validación. El modelo seleccionado tiene dimensión 128, cuatro cabezas de atención, tres capas de encoder, dimensión interna de 256 y 633,404 parámetros.

Los resultados muestran que ampliar la arquitectura no garantizó una mejor solución que ajustar la regularización. Asimismo, stride 4 ofreció una alternativa de menor costo con un error de validación cercano al de la configuración seleccionada.

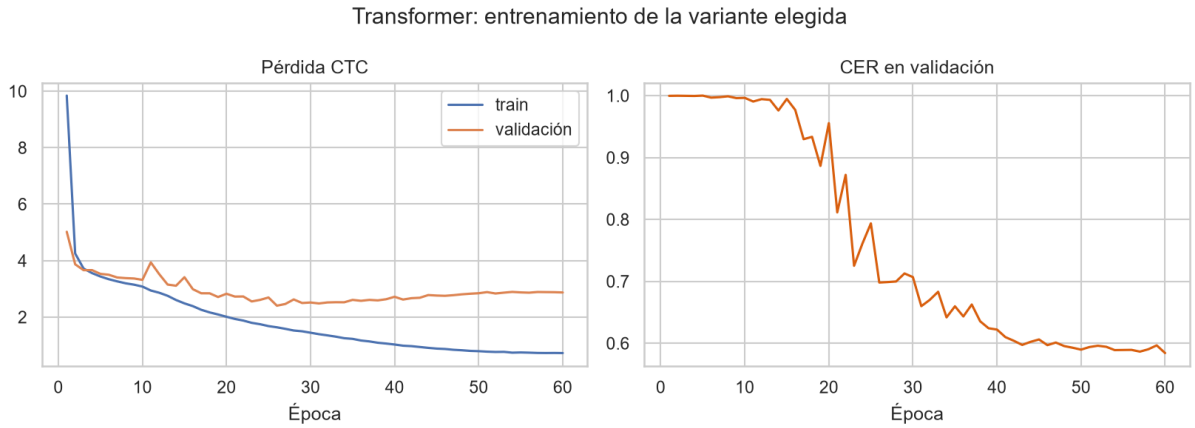



Figura 7. Resultados del ajuste de hiperparámetros del Transformer.

**Comparación de los modelos**

Los modelos seleccionados se evaluaron sobre las mismas 268 secuencias de prueba.

| **Indicador** | **CNN-BiGRU** | **Transformer** |
|---|---|---|
| CER de validación | 0.5516 | 0.5842 |
| CER de prueba | 0.4898 | 0.4991 |
| Complemento 1 − CER de prueba | 0.5102 | 0.5009 |
| Parámetros entrenables | 1,696,060 | 633,404 |
| Épocas ejecutadas | 60 | 60 |
| Tiempo de entrenamiento | 47.66 min | 17.81 min |
| Tiempo medio registrado por secuencia | 5.34 ms | 1.77 ms |
| Dispositivo registrado | CPU | CPU |

CNN-BiGRU obtuvo el menor CER de prueba, con una diferencia aproximada de 0.0094 respecto al Transformer. El Transformer utilizó casi el 37 % de sus parámetros y presentó menores tiempos registrados.

El tiempo por secuencia se obtuvo dividiendo la duración de la evaluación entre el número de ejemplos. El procedimiento incluye operaciones de evaluación, como pérdida, decodificación y métricas, por lo que no representa exclusivamente la latencia de una predicción individual ni el tiempo completo de interacción con la aplicación.

El análisis pareado documentado reporta que el Transformer obtuvo menor error en 103 secuencias, mayor error en 105 y el mismo error en 60. La prueba de Wilcoxon produjo un valor p cercano a 0.21. Por tanto, ese análisis no aportó evidencia suficiente para declarar una diferencia estadísticamente significativa; tampoco demuestra equivalencia entre los modelos.

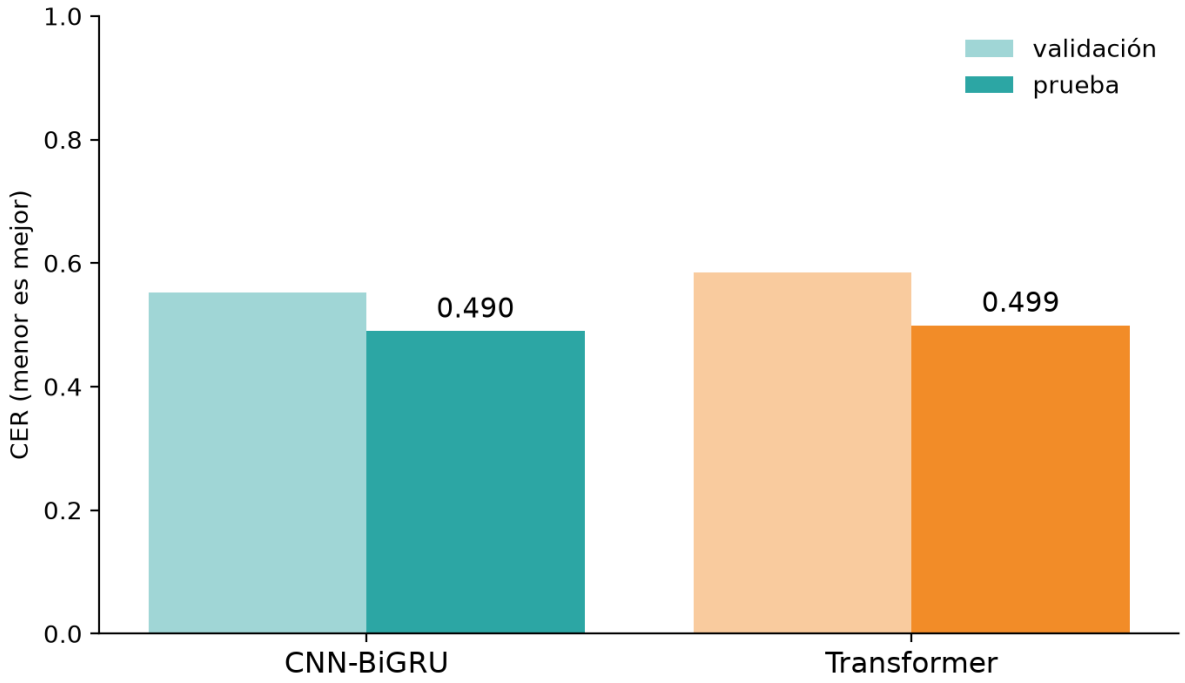



Figura 9. Comparación del CER de los modelos disponibles. Se recomienda conservar colores diferenciados y consistentes entre todas las figuras.

**Análisis de errores**

Los análisis guardados muestran que las eliminaciones de caracteres representan aproximadamente el 76 % de los errores de CNN-BiGRU y el 68 % de los del Transformer. En consecuencia, una dificultad importante consiste en producir transcripciones incompletas.

La cobertura de detección de las manos presenta una asociación negativa con el CER: aproximadamente −0.72 para CNN-BiGRU y −0.74 para Transformer, según los análisis de Spearman documentados. Las secuencias con mayor disponibilidad de información tienden a tener menor error, aunque esta asociación no establece causalidad.

También se observa una relación entre baja frecuencia de entrenamiento y mayor error por carácter. Este resultado respalda la necesidad de analizar el desempeño desagregado, dado que un promedio global puede ocultar dificultades en caracteres poco representados.

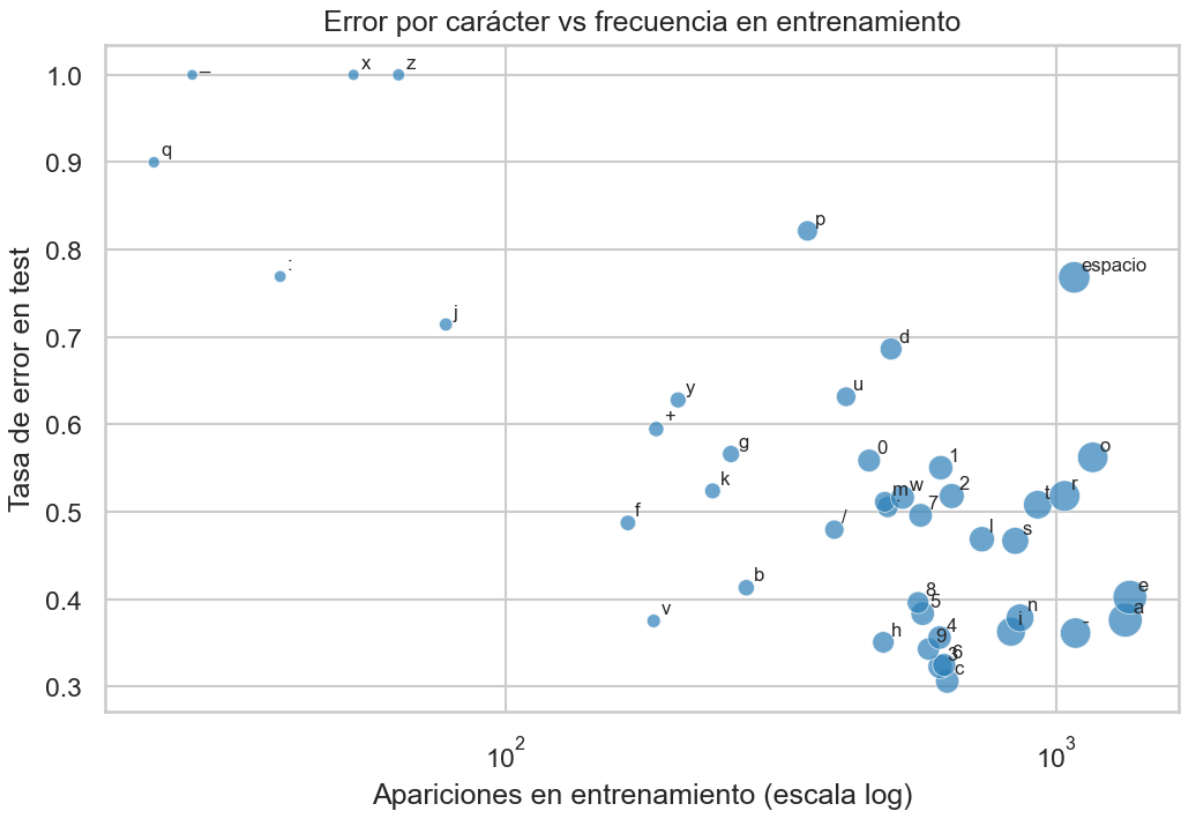



Figura 11. Relación entre frecuencia de entrenamiento y error por carácter en CNN-BiGRU.

Los errores elevados de ambos modelos indican que el prototipo todavía presenta limitaciones para producir transcripciones confiables de manera general. Los resultados demuestran aprendizaje de patrones, pero no justifican considerarlo un sistema de transcripción terminado.

**Aplicación desarrollada**

Se implementó una aplicación en Streamlit con páginas de inicio, predicción y rendimiento. Su propósito es permitir la exploración de los resultados sin ejecutar directamente los notebooks.

La página de inicio presenta el problema y los modelos disponibles. La página de predicción permite seleccionar ejemplos de prueba o cargar archivos Parquet compatibles. Para archivos nuevos se reutiliza el preprocesamiento del proyecto, incluyendo normalización, tratamiento de faltantes e indicadores de detección.

Cuando se dispone de una transcripción de referencia, la aplicación permite comparar el texto esperado con la predicción y mostrar errores mediante alineación de caracteres. En archivos sin referencia puede presentarse la predicción, pero no calcularse un CER verificable.

La página de rendimiento permite consultar métricas, curvas de entrenamiento, errores por carácter y comparaciones de costo. Streamlit facilita la integración con Python, mientras Plotly proporciona visualizaciones interactivas.

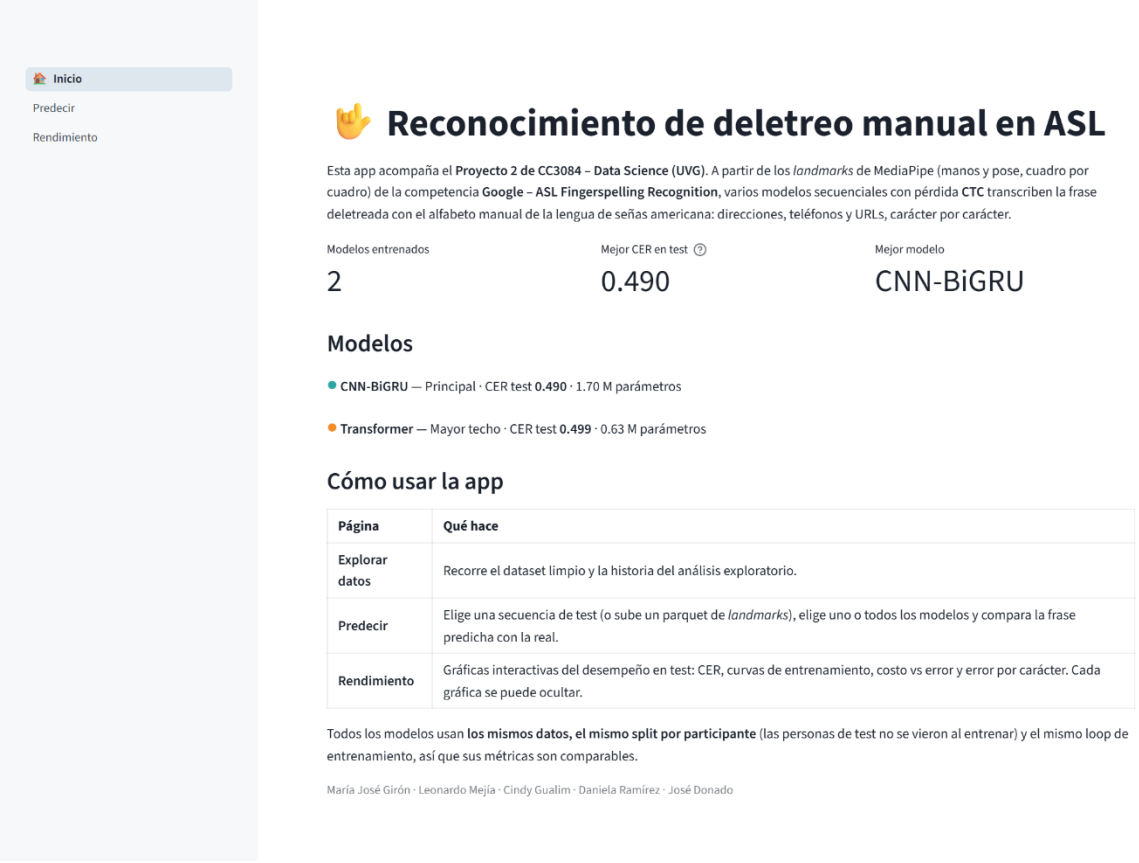



Figura 12. Página de inicio de la aplicación.

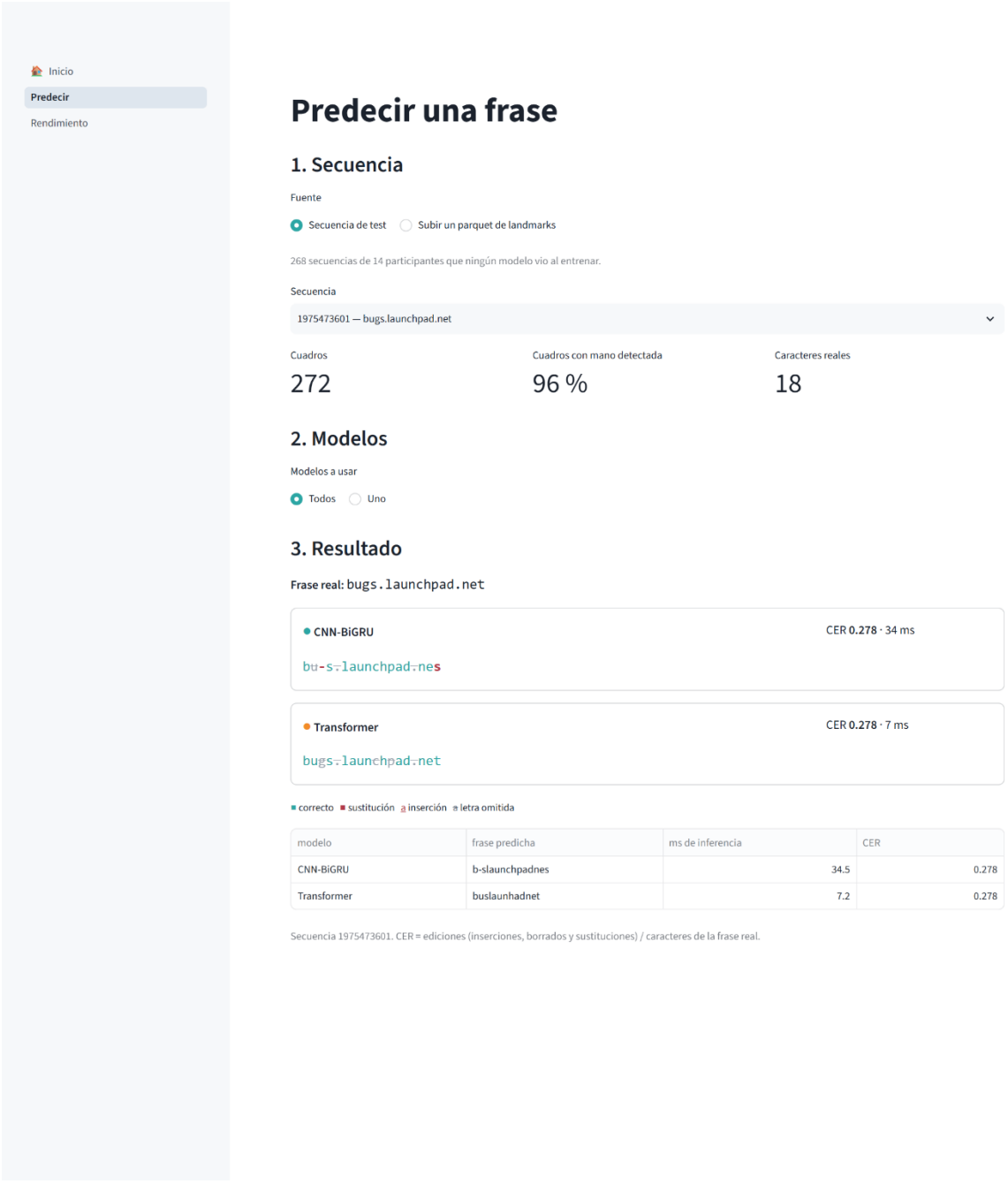



Figura 14. Interfaz para consultar o generar predicciones.

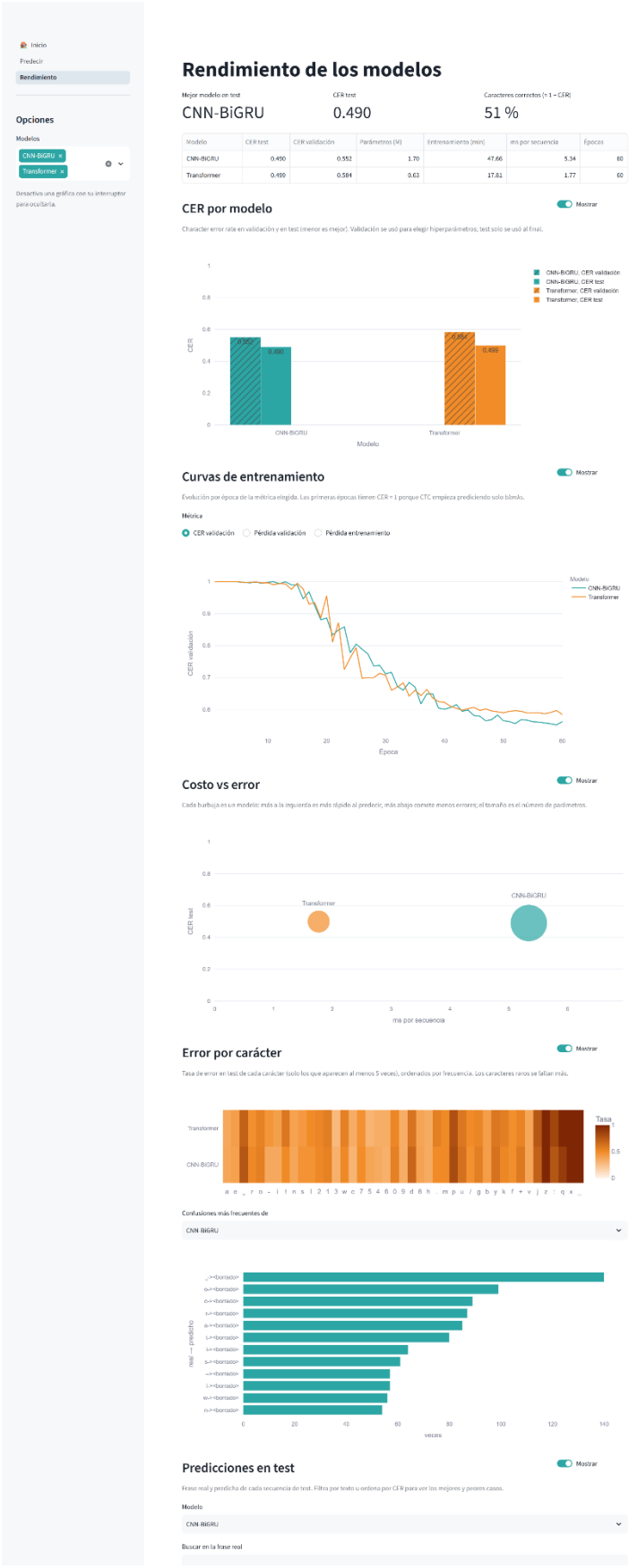



Figura 15. Interfaz de comparación de rendimiento.

## CONCLUSIONES

Durante la fase de evaluación se cumplió con el objetivo general de desarrollar y evaluar el prototipo de transcripción de deletreo manual de ASL a partir de secuencias de landmarks (i.e., los puntos de referencia que indican la parte de la mano). En total, se entrenaron, evaluaron y compararon cuatro modelos de aprendizaje automático con tasas de error (CER) relativamente bajas al evaluarlas entre sí, pero con una evidente oportunidad de mejora.

La caracterización del conjunto permitió identificar los siguientes desafíos: ausencia de detección en las manos, diferencias entre participantes por piel e iluminación, duración de secuencia diferente y desbalance en la frecuencia de los caracteres. Con dichos hallazgos, se fundamentaron las decisiones de preprocesamiento para interpretar y medir el error de todos los modelos. Las secuencias con mayor cobertura de detección de manos presentaron menos error, lo que resalta la asociación entre disponibilidad de información de entrada y el desempeño.

La preparación de los datos para el modelado se alcanzó mediante selección de variables, normalización anatómica, tratamiento de valores faltantes e incorporación de indicadores de detección. El filtrado conservó 1,878 secuencias de las 1,997 examinadas, representadas mediante 227 características por cuadro. Este procedimiento permitió procesar entradas de longitud variable, aunque la imputación no permitió recuperar los movimientos que dejaron de detectarse.

El objetivo de preparar los datos para el modelado se alcanzó mediante selección de variables, normalización anatómica, tratamiento de valores faltantes e incorporación de indicadores de detección. El filtrado conservó 1,878 secuencias de las 1,997 examinadas, representadas mediante 227 características por cuadro. Este procedimiento permitió procesar entradas de longitud variable, aunque la imputación no recupera los movimientos que dejaron de detectarse.

La división por participante permitió evaluar la capacidad de generalización a personas no observadas durante el entrenamiento. Se utilizaron 1,315 secuencias para entrenamiento (70% del total), 295 para validación (15%) y 268 (15%) para prueba, sin compartir participantes entre particiones para evitar fuga de información. Con dicha estrategia se redujo el riesgo de obtener resultados artificialmente favorables por reconocer estilos individuales ya presentes en entrenamiento. No obstante, el conjunto de prueba comprende únicamente 14 participantes, lo que limita el alcance de las conclusions y es importantísimo tener en cuenta para extrapolar los resultados a un experimento de muestra mayor.

La comparación de algoritmos mostró que CNN-BiGRU obtuvo el menor CER de prueba, con 0.4898, lo cual es favorable ya que significa menos error, seguido por Transformer con 0.4991, GRU con 0.5366 y LSTM con 0.5658. Las arquitecturas CNN-BiGRU y Transformer superaron a los modelos recurrentes unidireccionales en esta evaluación. Los resultados son compatibles con una ventaja de combinar información temporal más amplia, aunque no permiten atribuir la mejora exclusivamente al contexto bidireccional, porque también existen diferencias de arquitectura e hiperparámetros.

El ajuste de hiperparámetros demostró que aumentar la complejidad no garantiza mejores resultados. La configuración de mayor capacidad favoreció a CNN-BiGRU, mientras que en Transformer resultó más efectivo reducir el dropout. Para LSTM y GRU, las configuraciones seleccionadas utilizaron stride 4 y una paciencia que permitió completar las 60 epochs. Por lo tanto, la selección de parámetros debe considerar tanto el error de validación, como la estabilidad del aprendizaje y el costo computacional.

El Transformer presentó un equilibrio favorable entre calidad y costo frente a CNN-BiGRU: utilizó aproximadamente el 37 % de sus parámetros y requirió solo 17.81 minutos de entrenamiento, frente a 47.66 minutos. A criterio del grupo, un menor tiempo de entrenamiento es preferible a menor error cuando se trata de proyectos educativos que requieren un deployment sencillo. Sin embargo, para casos de uso real puede resultar más valioso una reducción en el error por ínfima que sea dado a que el costo de oportunidad es traducir mal una palabra y llegar a cambiar el significado de la idea original por completo.

La comparación de Wilcoxon entre estos produjo un valor p cercano a 0.21, por lo que no se encontró una diferencia estadísticamente significativa en el análisis realizado. Esto respalda considerar al Transformer como una alternativa eficiente para el prototipo, sin afirmar que ambos modelos sean equivalentes.

Con el análisis de errores se encontró que hay caracteres que se pierden en la traducción para las cuatro arquitecturas. Además, el desempeño varía por la frecuencia de caracteres y la cobertura de detección de manos, por lo que se sugiere explorar mejoras en la calidad y diversidad de datos, evaluar si es oportuno implementar la aumentación y utilizar otras estrategias de decodificación dado a que en el experimento contiguo no se midió el efecto de estos componentes.

**REFERENCIAS**


- Albanie, S., Varol, G., Momeni, L., Afouras, T., Chung, J.S., Fox, N., Zisserman, A.: Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues. In: European conference on computer vision. pp. 35–53. Springer (2020)
- Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv. https://arxiv.org/abs/1803.01271
- Becerril-Carrillo, A. d. J., Selley-Rojas, H. J., & Guevara-Martínez, E. (2026). Recurrent Neural Networks for Mexican Sign Language Interpretation in Healthcare Services. Sensors, 26(1), 27. https://doi.org/10.3390/s26010027
- Buribayev, Z., Aouani, M., Zhangabay, Z., Yerkos, A., Abdirazak, Z., & Zhassuzak, M. (2025). Enhancing Kazakh sign language recognition with BiLSTM using YOLO keypoints and optical flow. Applied Sciences, 15(10), Article 5685. https://doi.org/10.3390/app15105685
- Camgoz, N. C., Koller, O., Hadfield, S., & Bowden, R. (2020). Sign language transformers: Joint end-to-end sign language recognition and translation. arXiv. https://arxiv.org/abs/2003.13830
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (pp. 1724–1734). Association for Computational Linguistics. https://aclanthology.org/D14-1179
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv. https://arxiv.org/abs/1412.3555
- Fayyazsanavi, P., Nejatishahidin, N., & Košecká, J. (2023). Fingerspelling PoseNet: Enhancing fingerspelling translation with pose-based transformer models [Preprint]. arXiv. https://arxiv.org/abs/2311.12128
- Georg, M., Tanzer, G., Uboweja, E., Hassan, S., Shengelia, M., Sepah, S., Forbes, S., & Starner, T. (2025). FSboard: Over 3 million characters of ASL fingerspelling collected via smartphones. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 13897–13906). https://openaccess.thecvf.com/content/CVPR2025/html/Georg_FSboard_Over_3_Million_Characters_of_ASL_Fingerspelling_Collected_via_CVPR_2025_paper.html
- Google. (2023). American Sign Language fingerspelling recognition [Competencia]. Kaggle. https://www.kaggle.com/competitions/asl-fingerspelling
- Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural

networks. In Proceedings of the 23rd International Conference on Machine Learning (pp. 369–376). https://www.cs.toronto.edu/~graves/icml_2006.pdf

- Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM and other neural network architectures. Neural Networks, 18(5–6), 602–610. https://archivedsite.air.in.tum.de/Main/Publications/Graves2005a.pdf
- Hannun, A. (2017). Sequence modeling with CTC. Distill, 2(11). https://doi.org/10.23915/distill.00008
- Henkel, C. (2023). 1st place solution to the Google – American Sign Language Fingerspelling Recognition competition [Repositorio de código]. GitHub. https://github.com/ChristofHenkel/kaggle-asl-fingerspelling-1st-place-solution
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735
- Hyunchul Lim, Nam Anh Dang, Dylan Lee, Tianhong Catherine Yu, Jane Lu, Franklin Mingzhe Li, Yiqi Jin, Yan Ma, Xiaojun Bi, François Guimbretière, and Cheng Zhang. (2025). SpellRing: Recognizing Continuous Fingerspelling in American Sign Language using a Ring. In Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25). Association for Computing Machinery, New York, NY, USA, Article 67, 1–17. https://doi.org/10.1145/3706598.3713721
- Kamble, S. (2025). SLRNet: A real-time LSTM-based sign language recognition system. arXiv. https://arxiv.org/abs/2506.11154
- Kim, S., Gholami, A., Shaw, A., Lee, N., Mangalam, K., Malik, J., Mahoney, M. W., & Keutzer, K. (2022). Squeezeformer: An efficient transformer for automatic speech recognition. Advances in Neural Information Processing Systems, 35, 9361–9373. https://arxiv.org/abs/2206.00888
- Korotaev, P., Surovtsev, P., Kapitanov, A., Kvanchiani, K., & Nagaev, A. (2025). HandReader: Advanced techniques for efficient fingerspelling recognition. arXiv. https://arxiv.org/abs/2505.10267
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324. https://doi.org/10.1109/5.726791
- Myagila K, Nyambo DG and Dida MA (2025) Efficient spatio-temporal modeling for sign language recognition using CNN and RNN architectures. Front. Artif. Intell. 8:1630743. doi: 10.3389/frai.2025.1630743
- NVIDIA. (2023, 23 de octubre). Reading the signs: NVIDIAN wins Kaggle AI contest for sign language recognition [Entrada de blog]. NVIDIA Blog. https://blogs.nvidia.com/blog/2023/10/23/nvidia-life-kaggle-ai-contest-sign-language-recognition/

- Olah, C. (2015, 27 de agosto). Understanding LSTM networks [Entrada de blog]. colah's blog. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
- Padden, C. A., & Gunsauls, D. C. (2003). How the alphabet came to be used in a sign language. Sign Language Studies, 4(1), 10–33.
- Patil, D., Patil, A., Patil, A., More, A., & Patil, S. S. (2024). Sign Language to Text Conversion. International Journal for Research in Applied Science and Engineering Technology. https://doi.org/10.22214/ijraset.2024.62147
- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. IEEE Transactions on Signal Processing, 45(11), 2673–2681. https://doi.org/10.1109/78.650093
- Shi, B., Brentari, D., Shakhnarovich, G., & Livescu, K. (2021). Fingerspelling Detection in American Sign Language. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4164–4173. https://doi.org/10.1109/cvpr46437.2021.00415
- Shi, B., Del Rio, A. M., Keane, J., Brentari, D., Shakhnarovich, G., & Livescu, K. (2019). Fingerspelling recognition in the wild with iterative visual attention. In Proceedings of the IEEE/CVF International Conference on Computer Vision. https://arxiv.org/abs/1908.10546
- Srivastava, S., Singh, S., Pooja, & Prakash, S. (2024). Continuous sign language recognition system using deep learning with MediaPipe Holistic. arXiv. https://arxiv.org/abs/2411.04517
- Subramanian, B., Olimov, B., Naik, S. M., Kim, S., Park, K.-H., & Kim, J. (2022). An integrated mediapipe-optimized GRU model for Indian sign language recognition. Scientific Reports, 12, Article 11964. https://doi.org/10.1038/s41598-022-15998-7
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30. https://arxiv.org/abs/1706.03762
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into deep learning. Cambridge University Press. https://d2l.ai
- Zhang, Y., & Jiang, X. (2024). Recent Advances on Deep Learning for Sign Language Recognition. Computer Modeling in Engineering & Sciences, 139(3), 2399–2450. https://doi.org/10.32604/cmes.2023.045731
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., y Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. arXiv. https://arxiv.org/abs/2006.10214

Fuente propia de los resultados experimentales: notebooks, código, archivos de métricas y figuras del repositorio CC3084-Proyecto2 se encuentran publicadas y pueden consultarse en este enlace: https://github.com/DufreyM/CC3084-Proyecto2